НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ «ОДЕСЬКА ПОЛІТЕХНІКА»
МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
ІНСТИТУТ КОМП'ЮТЕРНИХ СИСТЕМ
КАФЕДРА ІНФОРМАЦІЙНИХ СИСТЕМ

Протокол лабораторної роботи №1
«Розгортання середовища та порівняльний аналіз мовних моделей»
з дисципліни: «Обчислювальні методи»

Виконав(-ла): студент групи АІ-241
Сівак Вадим
Олександрович

Перевірив: ас. Баранов О.М.

Одеса – 2026

Мета роботи

Запустити мовну модель локально й керувати нею з коду; переконатися, що локальна й хмарна моделі мають єдиний інтерфейс; виміряти продуктивність інференсу й пояснити, з чого складається затримка; емпірично показати вплив temperature і top_p на стабільність відповідей.

Варіант 7 (номер у журналі 17; за вказівкою викладача 17 mod 10 = 7).

Конфігурація середовища

Таблиця Д1 – Обладнання та програмне забезпечення

Компонент	**Значення**
Процесор	Intel Core Ultra 9 275HX
Відеокарта	NVIDIA GeForce RTX 5070 Laptop GPU, 8 ГБ VRAM (драйвер 616.64)
Оперативна пам'ять	32 ГБ (доступно 31,4 ГБ)
ОС	Windows 11 Pro 10.0.26300
Ollama	0.40.3 (встановлено через winget, служба на http://localhost:11434)
Python	3.12.10, venv; openai 3.29.0, python-dotenv 1.2.4, matplotlib 3.11.2
Локальні моделі	qwen3:4b (2,5 ГБ), llama3.2:3b (2,0 ГБ), qwen3:8b (5,2 ГБ) – RAM ≥ 16 ГБ

Усі локальні моделі виконуються на відеокарті: ollama ps показує PROCESSOR = 100% GPU. Тому швидкість генерації (85–120 токенів/с) значно вища за «кілька слів за секунду», описані в методичці для інференсу на процесорі.

Усі запуски журналюються логером custom_logger у файл lab1.log; у лог потрапили лише фінальні запуски (налагодження виконувалось в окремій копії скриптів).

Крок 2. Модель – це звичайний веб-сервер

Запит POST /api/generate до qwen3:4b з промптом «Столиця України?» (скрипт task1_generate.py, аналог curl із методички) повернув:

Таблиця Д2 – Поля відповіді /api/generate (qwen3:4b, «Столиця України?»)

Поле	**Значення**	**Пояснення**
response	Київ	текст, який бачить користувач
thinking	4954 символи (англійською)	внутрішня чернетка: модель кілька разів перепитує себе, чи не Kiev

done / done_reason	True / stop	модель сама завершила відповідь
context	1213 токенів: [151644, 872, 198, …]	діалог у вигляді номерів токенів
load_duration	17 087 400 нс = 0,017 с	модель уже була в пам'яті
prompt_eval_count / cached	19 / 4	≈6–7 токенів тексту, решта – службова розмітка; 4 з кешу
prompt_eval_duration	0,029 с	обробка запиту (prefill)
eval_count / eval_duration	1195 / 13,50 с	генерація разом із роздумами (88,5 токенів/с)
total_duration	13,60 с	усе разом

Завдання 1. Ціна роздумів: міркувальна vs інструктивна модель

Виконано запити до двох моделей через ендпойнт /api/generate (task1_generate.py). Перед серією вимірів зроблено прогрівальний запит до кожної моделі, тому load_duration не впливає на час. Спочатку – просте фактичне питання, потім – задача на міркування (5 запусків для кожної моделі).

Таблиця 1 – Фактичне питання «Столиця України? Відповідай одним словом.»

Показник	**qwen3:4b**	**llama3.2:3b**
response	«…правильный ответ на русском языке – **Киев**» (3 абзаци російською)	Київ.
eval_count	4614	5
Час, с	55,05	0,09
Токенів/с	84,02	123,20

Міркувальна модель згенерувала в 923 рази більше токенів і витратила в 612 разів більше часу. Її чернетка (13 126 символів російською) – це розмірковування, якою мовою відповідати («Київ» чи «Киев»); у підсумку вона порушила і вимогу «одним словом», і мову питання. У налагоджувальному прогоні та сама модель відповіла «Kyiv» за 317 токенів – обсяг роздумів на тому самому питанні змінюється в рази від запуску до запуску.

Таблиця 2 – Задача на міркування (середнє з 5 запусків), правильна відповідь – 5 грн

Показник	**qwen3:4b**	**llama3.2:3b**
Правильних із 5	5 / 5	2 / 5
eval_count	1327,0	276,0

Час, с	15,14	2,61
Токенів/с	88,80	110,53

Відповіді llama3.2:3b у трьох хибних запусках: 90 грн, 10 грн і 50 грн. Типова помилка – «10 грн» (запуск 3): модель просто відняла 110 − 100, не врахувавши, що різниця 100 грн стосується обох доданків. Крім того, llama розписувала розв'язання російською, хоча питання українською. qwen3:4b у всіх п'яти запусках склала рівняння x + (x + 100) = 110 і перевірила відповідь.

Відповіді на питання (крок 6, чат-бот Claude)

1. Я порівнював міркувальну модель (qwen3:4b) та інструктивну (llama3.2:3b) на простому фактичному питанні. Відповідь обидві дали однакову, але міркувальна згенерувала в десятки разів більше токенів. Куди пішли ці токени?

Відповідь: Майже всі вони пішли в поле thinking – внутрішню чернетку, у якій модель переформульовує питання, перебирає варіанти й кілька разів перепровіряє себе. Ці токени генеруються з тією ж швидкістю, що й відповідь, займають контекст і в платних API оплачуються як вихідні, хоча користувач бачить лише одне слово в response.

2. На яких типах задач міркувальні моделі дають перевагу над інструктивними, а на яких їх використання невиправдане?

Відповідь: Перевага – на багатокрокових задачах: арифметика й логіка з «пасткою інтуїції», планування, код, перевірка умов, де проміжні результати треба десь «записати». Невиправдано – на фактичних питаннях, переказі, класифікації, генерації шаблонного тексту і в сценаріях, критичних до затримки (чат, агент із багатьма послідовними викликами).

3. Чим відрізняється процес навчання міркувальної моделі від інструктивної?

Відповідь: Інструктивну модель донавчають на парах «інструкція – відповідь» (SFT) і вирівнюють за вподобаннями людей (RLHF/DPO). Міркувальну додатково вчать генерувати довгі ланцюжки міркувань: SFT на трасах міркувань

(часто дистильованих із більшої моделі) і навчання з підкріпленням із винагородою за правильну кінцеву відповідь у перевірюваних задачах (математика, код), тож модель «вчиться думати довше», коли це підвищує точність. Qwen3 – гібрид: режим міркування можна вмикати й вимикати.

4. Чи означає більша кількість згенерованих токенів у міркувальної моделі, що вона «розумніша» за інструктивну?

Відповідь: Ні: кількість токенів – це витрати, а не якість. Якість вимірюється правильністю відповіді на конкретному типі задач; на фактичному питанні обидві моделі відповіли за змістом однаково, а перевага qwen3 на задачі про ручку пояснюється тим, що розписаний хід став для неї робочою пам'яттю. Довге міркування може й нашкодити – модель здатна «передумати» правильну відповідь.

5. Як пояснити різницю у швидкості генерації (токенів за секунду) між моделями qwen3:4b та llama3.2:3b?

Відповідь: Генерація одного токена потребує прочитати з пам'яті всі ваги моделі, тож швидкість обмежена пропускною здатністю пам'яті й розміром моделі: qwen3:4b має ~4 млрд параметрів (2,5 ГБ у Q4) проти ~3,2 млрд (2,0 ГБ) у llama3.2:3b, тобто більше обчислень і читань на токен. Додатково впливають архітектура (кількість шарів, словник qwen3 ~152 тис. токенів) і те, що довгий контекст міркувань сповільнює кожен наступний крок уваги.

Висновок за завданням 1

Експеримент показав, що два показники – швидкість генерації та загальний час відповіді – не можна плутати. Швидкість у двох моделей близька (88,8 проти 110,5 токенів/с) і характеризує обладнання та розмір моделі, а загальний час відрізняється на порядки й характеризує обсяг роботи, яку модель собі призначила. На фактичному питанні режим міркування виявився чистою перевитратою: результат не кращий (і навіть гірший за форматом), а витрати більші в 612 разів за часом. На задачі про ручку й зошит ситуація протилежна – розписаний хід дав 5 правильних із 5 проти 2 із 5. Практичний висновок: тип моделі треба обирати під

тип задачі; для агента з 8 послідовними викликами різниця складе ≈121 с (8 × 15,1) проти ≈21 с (8 × 2,6), що робить міркувальну модель незручною для налагодження.

Завдання 2. Роль system-повідомлення: керування поведінкою моделі

Питання варіанта 7: «Поясни різницю між процесом і потоком.» Скрипт hello.py надсилає його обом моделям із двома system-повідомленнями (temperature = 0,7) і логує відповідь та response.usage.

Таблиця 3 – Вплив system-повідомлення

Модель	**System**	**Довжина, симв.**	**Мова відповіді**	**Речень**	**Інструкцію виконано?**
llama3.2:3b	лаконічний асистент	241	українська з русизмами й латиницею («фіксирована», «електроенергіі»)	3	частково: коротко, але не по суті – описано фізичний «потік»
llama3.2:3b	українською, 2 речення	239	українська з англ. вкрапленням («refers»)	2	ні: два речення, але не лише українською, зміст хибний («серії зображень»)
qwen3:4b	лаконічний асистент	166	українська	2	так
qwen3:4b	українською, 2 речення	216	українська (дрібні помилки: «відведений пам'ять»)	2	так

Таблиця 4 – Витрати токенів

Модель	**System**	**prompt_tokens**	**completion_tokens**	**total_tokens**
llama3.2:3b	лаконічний асистент	50	81	131
llama3.2:3b	українською, 2 речення	54	76	130
qwen3:4b	лаконічний асистент	53	2064	2117
qwen3:4b	українською, 2 речення	62	572	634

Відповіді на питання (крок 6)

1. Яка модель точніше дотрималась обмеження «максимум двома реченнями»?

Відповідь: qwen3:4b: в обох випадках рівно два речення, і вони по суті. llama3.2:3b формально вклалася в два речення, але відповідь беззмістовна, тож обмеження виконано лише за формою.

2. Чи були випадки, коли модель відповіла не українською, попри пряму вказівку?

Відповідь: Так: llama3.2:3b навіть із вимогою «лише українською» вставила англійське «refers», а в першій відповіді – русизм «фіксирована» й латиницю «електроенергіі». qwen3:4b відповідала українською.

3. Як змінився prompt_tokens при зміні system-повідомлення і чому?

Відповідь: Зріс: у llama 50 → 54, у qwen 53 → 62, бо друге system-повідомлення довше. Службова розмітка ролей не змінилася, а різний приріст пояснюється різними токенізаторами моделей.

4. Чому в qwen3:4b значення completion_tokens значно більше, ніж у llama3.2:3b, навіть при однаковій довжині видимої відповіді?

Відповідь: Бо в completion_tokens враховано приховані міркування: OpenAI-сумісний ендпойнт Ollama повертає їх в окремому полі reasoning, а не в content. 166 видимих символів коштували 2064 токени – у 25 разів більше, ніж повна відповідь llama.

Відповіді на питання (крок 7, чат-бот Claude)

1. Я порівнював інструктивну модель (llama3.2:3b) та міркувальну (qwen3:4b) з різними system-повідомленнями. Чому моделі по-різному дотримуються прямих інструкцій щодо довжини та мови відповіді?

Відповідь: Дотримання інструкцій – навичка, набута на етапі донавчання, і в різних моделей вона різна: залежить від даних SFT/RLHF, частки багатомовних прикладів і розміру моделі. Обмеження на довжину вимагає «планувати» відповідь наперед, а мовне обмеження конкурує з тим, якою мовою модель найчастіше бачила схожі тексти; міркувальна модель ще й встигає «нагадати» собі інструкцію в чернетці.

2. Чим system-повідомлення відрізняється від user-повідомлення з точки зору моделі — чи справді воно має більший пріоритет?

Відповідь: Для моделі це той самий текст у контексті, лише обгорнутий спеціальними токенами ролі. Пріоритет system з'являється тільки тому, що модель так навчили (ієрархія інструкцій), тому він статистичний, а не гарантований: малі моделі часто зважають на нього слабше, ніж на останнє user-повідомлення.

3. Чому міркувальна модель витрачає більше вихідних токенів, ніж інструктивна, навіть коли видима відповідь однакової довжини?

Відповідь: Бо completion_tokens враховує і приховану чернетку reasoning/thinking, яку OpenAI-сумісний ендпойнт Ollama не показує в content, але генерує й рахує. Видима відповідь – лише хвіст значно довшого виводу.

4. Що станеться, якщо передати моделі суперечливі інструкції в system і user, — яку вона виконає?

Відповідь: Однозначної відповіді немає: добре вирівняні моделі частіше дотримуються system, але малі моделі нерідко виконують останню (user) інструкцію або змішують обидві. Тому критичні обмеження не варто довіряти лише промпту – їх перевіряють у коді.

5. Чому малі мовні моделі часто гірше дотримуються інструкцій щодо мови відповіді, ніж великі?

Відповідь: У малих моделей менше «місткості» на рідкісні мови: українських текстів у навчанні на порядки менше, ніж англійських і російських, тому під час генерації модель «зісковзує» в мову, яку знає краще. Великі моделі мають достатньо параметрів, щоб утримувати і мову, і зміст одночасно.

Висновок за завданням 2

System-повідомлення – найдешевший спосіб керувати моделлю, але ненадійний: воно лише зсуває ймовірності, а не гарантує результат. Мала інструктивна модель дотрималася кількості речень, але порушила мову й зміст; міркувальна модель виконала інструкцію, проте ціною в 7–25 разів більшої кількості вихідних токенів. Самої інструкції недостатньо, якщо формат критичний:

відповідь треба перевіряти програмно (кількість речень, мова, JSON-схема) і при порушенні повторювати запит із поясненням помилки, знижувати temperature, давати приклад бажаної відповіді або брати більшу модель.

Завдання 3. Локальна vs хмарна модель: єдиний інтерфейс

Хмарний провайдер – Groq (безкоштовний тариф, картка не потрібна): base_url = https://api.groq.com/openai/v1. Модель із методички llama-3.3-70b-versatile у Groq на момент виконання вже недоступна (у списку client.models.list() її немає), тому обрано найбільшу доступну модель – openai/gpt-oss-120b (120 млрд параметрів, MoE, міркувальна). Ключ зберігається у файлі .env (CLOUD_API_KEY) і зчитується load_dotenv(); .env і venv/ додано в .gitignore, git status файл .env не показує. Питання варіанта 7: «Поясни різницю між процесом і потоком. Коротко.» (compare.py).

```
PROVIDERS = {
    "local": {"base_url": "http://localhost:11434/v1", "api_key": "ollama",
              "model": "llama3.2:3b"},
    "cloud": {"base_url": "https://api.groq.com/openai/v1",
              "api_key": os.getenv("CLOUD_API_KEY"),
              "model": "openai/gpt-oss-120b"},
}
```

Таблиця 5 – Порівняння локальної та хмарної моделі

Показник	**Локальна**	**Хмарна**
Назва моделі	llama3.2:3b	openai/gpt-oss-120b (Groq)
Час відповіді, с	3,41	0,70
completion_tokens	142	266 (з них reasoning 37)
Довжина відповіді, символів	431	685
Мова відповіді	українська з помилками й русизмами («родинськими», «одиноho», «зустрічка»)	українська, грамотна (дрібна помилка «діляє»)
Фактичних помилок у відповіді	відповідь хибна повністю (≥3: «потік – серія зустрічей у лікарні», процес – «зведення», «відмінності різкі»)	0

Відповіді на питання (крок 5)

1. Скільки рядків коду довелося змінити, щоб перейти з локальної моделі на хмарну?

Відповідь: Два – base_url і model (плюс api_key, який береться з .env). Функція create() і решта compare.py для обох провайдерів однакові: змінюється лише вміст словника PROVIDERS.

2. Яка відповідь повніша й точніша за змістом? Наведіть конкретний приклад різниці.

Відповідь: Хмарна: вона пояснює, що процес має власний адресний простір і ресурси, а потоки всередині процесу ділять спільну пам'ять, тому обмінюються даними швидко, але можуть заважати один одному. Локальна модель сплутала терміни з побутовими значеннями: «потік – це серія зустрічей людей у лікарні».

3. Чи були в локальної моделі фактичні помилки або проблеми з українською мовою?

Відповідь: Так, і те, і те: зміст повністю хибний, а текст містить русизми й неіснуючі слова («родинськими», «зустрічка», «одиноho» з латинською o).

4. Хмарна модель відповіла швидше за локальну — за рахунок чого, якщо запит летить через інтернет?

Відповідь: Groq виконує модель на спеціалізованих прискорювачах LPU з дуже швидкою пам'яттю, тому генерація (≈480 токенів/с за completion_time 0,56 с) у рази швидша за ноутбучний GPU; мережа додала лише десяті частки секунди. Локальна модель до того ж втрачає ~2 с на з'єднання з localhost (див. завдання 4, табл. Д3), а її відповідь містила більше «води».

Відповіді на питання (крок 6, чат-бот Claude)

1. Чому формат запитів OpenAI став стандартом, який підтримують інші провайдери, зокрема локальний сервер Ollama?

Відповідь: OpenAI першою отримала масову аудиторію розробників, і під її API написали SDK, бібліотеки (LangChain, LlamaIndex) та тисячі застосунків. Новому провайдеру вигідніше прийняти той самий формат, ніж змушувати всіх переписувати код, – це мережевий ефект і низька вартість переходу для клієнтів.

2. Я порівнював локальну модель на 3 млрд параметрів і хмарну на 120 млрд. Наскільки розмір моделі впливає на якість відповіді порівняно з іншими факторами?

Відповідь: Розмір – найсильніший окремий фактор для фактичних знань, мовної якості й багатокрокових міркувань: модель на 120B просто «пам'ятає» більше. Але якість даних і донавчання, наявність режиму міркування, квантизація

та якість промпта можуть дати порівнянний ефект, а на простих задачах різниця між 3B і 120B мала.

3. Хмарна модель відповіла швидше за локальну, попри затримку мережі. Чим це пояснюється?

Відповідь: Хмарний провайдер запускає модель на спеціалізованому залізі (кластери GPU H100 або LPU у Groq) з пропускною здатністю пам'яті на порядок вищою за ноутбучну відеокарту, з оптимізованими ядрами та батчингом. Мережева затримка – десятки чи сотні мілісекунд, а виграш у швидкості генерації – сотні токенів за секунду.

4. Які ризики для приватності виникають при використанні хмарних API і що з цим роблять у комерційних проєктах?

Відповідь: Текст запиту покидає ваш контур: його можуть логувати, зберігати, переглядати модератори чи (на безкоштовних тарифах) використовувати для навчання; є ризики витоку й порушення GDPR/NDA. Комерційні проєкти укладають договори з умовою zero data retention, обирають регіон обробки, маскують персональні дані перед відправкою або розгортають модель у власній інфраструктурі.

5. Що таке rate limit у безкоштовних тарифах і як розробники обходять це обмеження?

Відповідь: Rate limit – ліміт запитів чи токенів за хвилину/добу, після якого сервер відповідає 429 Too Many Requests. Його легально обходять повторними спробами з експоненційною паузою, кешуванням відповідей, пакетною обробкою, перемиканням між кількома провайдерами (той самий OpenAI-сумісний код) або переходом на платний тариф.

Висновок за завданням 3

Перемикання між локальною й хмарною моделлю справді зводиться до зміни адреси, ключа й назви моделі – решта коду не знає, з ким працює. Локальну модель я обрав би, коли дані не можна передавати назовні (персональні, комерційні, закриті), потрібна робота без інтернету чи передбачувана нульова вартість, а задача

проста (класифікація, витяг із наданого тексту, чернетки). Хмарну – коли важливі якість, знання й мова (як у цьому завданні, де 3B-модель відповіла хибно), швидкість на довгих відповідях і немає обмежень на передачу даних; при цьому треба закладати ліміти (у нас 8000 токенів/хв) і резервного провайдера.

Завдання 4. Вимірювання швидкості: TTFT і залежність від довжини запиту

Промпти варіанта 7: короткий – «Столиця Норвегії?», середній – «Поясни в одному абзаці, що таке рекурсія.», довгий – «Напиши інструкцію з 10 пунктів, як провести код-рев'ю.» Вимірювання – benchmark.py (стрімінг, прогрів, 3 повтори, медіана).

Для хмарної моделі між вимірами додано паузу 20 с поза вимірюваним інтервалом: безкоштовний тариф Groq обмежує 8000 токенів/хв, і повтор після помилки 429 інакше потрапив би всередину виміру. TTFT рахується до першого видимого токена (content): міркувальна gpt-oss-120b спершу генерує короткий reasoning, і він входить у TTFT.

Таблиця 7 – Результати вимірювання швидкості (медіана з 3 повторів, results.csv)

Розміщення	**Модель**	**Промпт**	**TTFT, с**	**Всього, с**	**Симв./с**
локальна	llama3.2:3b	короткий	2,101	2,240	11,2
локальна	llama3.2:3b	середній	2,092	3,148	112,4
локальна	llama3.2:3b	довгий	2,113	7,963	229,6
хмарна	gpt-oss-120b	короткий	0,500	0,527	45,5
хмарна	gpt-oss-120b	середній	0,263	0,642	698,6
хмарна	gpt-oss-120b	довгий	0,390	2,853	1172,7

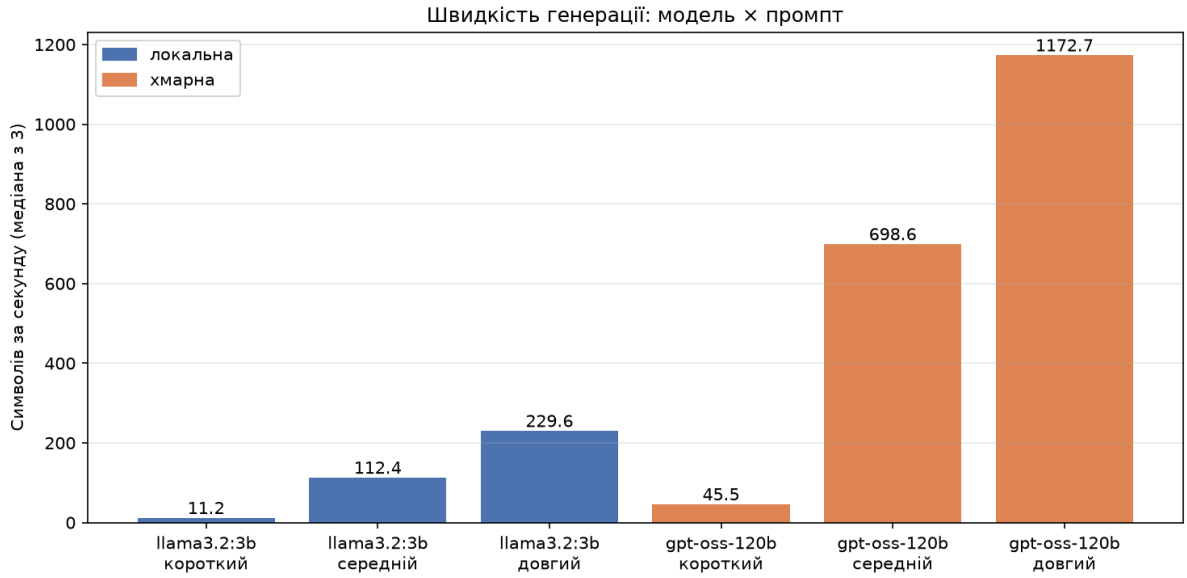


Рисунок 1 – Швидкість генерації (символів за секунду) для шести комбінацій «модель × промпт» (chart.py)

Таблиця 8 – Холодний і теплий старт локальної моделі llama3.2:3b (короткий промпт, cold_warm.py)

Показник	**Холодний старт**	**Теплий старт**
TTFT, с	4,053	2,070
Всього, с	4,171	2,188

Холодний старт виміряно одразу після ollama stop llama3.2:3b: близько 2 с пішло на читання 2 ГБ ваг із диска та розміщення їх у відеопам'яті.

Додатковий експеримент: з чого складаються ~2 с TTFT локальної моделі

TTFT теплої локальної моделі на GPU (≈2,07 с) виявився підозріло великим – стільки ж показує й приклад викладача (2,06 с). Причина – у мережевому стеку Windows: ім'я localhost резолвиться спершу в IPv6-адресу ::1, а Ollama слухає лише IPv4 127.0.0.1, тож кожне нове з'єднання ~2 с чекає на відкат до IPv4. Скрипт ttft_localhost.py порівнює дві адреси (медіана з 3, після прогріву):

Таблиця Д3 – TTFT llama3.2:3b залежно від адреси сервера

Адреса	**getaddrinfo**	**TTFT, с**	**Всього, с**
http://localhost:11434	::1, потім 127.0.0.1	2,077	2,208
http://127.0.0.1:11434	127.0.0.1	0,048	0,507

Отже, власне обробка запиту на GPU займає ~0,05 с, а 97 % «TTFT» – це спроба з'єднатися з IPv6. Для відповідності методичці основні виміри виконано з адресою localhost, але це треба враховувати при порівнянні з хмарою.

Відповіді на питання (крок 6)

1. У якої моделі кращий TTFT і чому саме в неї?

Відповідь: За таблицею 7 – у хмарної (0,26–0,50 с проти 2,09–2,11 с), всупереч очікуванню з методички. Причина – не швидкість моделі: локальний TTFT на 97 % складається з відкату localhost з IPv6 на IPv4; через 127.0.0.1 він

становить 0,048 с (табл. Д3) і тоді кращий у локальної моделі, бо їй не потрібна мережа, черга на спільному сервері й попередній reasoning.

2. Чи збігається «найшвидша за TTFT» з «найшвидшою за символами/с»? Якщо ні — поясніть.

Відповідь: У наших вимірах обидва показники кращі в хмарної моделі, але лише через артефакт localhost. Без нього лідери різні: за TTFT – локальна (0,05 с), за символами/с – хмарна (1173 проти 230 на довгому промпті), бо TTFT визначається з'єднанням і prefill, а швидкість – пропускною здатністю заліза під час генерації.

3. Як змінилося співвідношення локальної та хмарної моделі при переході від короткого промпту до довгого?

Відповідь: За загальним часом хмарна випереджає в 4,3 раза на короткому промпті (2,24 / 0,53 с) і в 2,8 раза на довгому (7,96 / 2,85 с), а за символами/с перевага зростає до 5–6 разів. Якщо прибрати 2 с затримки localhost, локальна модель виграла б короткий промпт (≈0,2 с проти 0,53 с) і програла довгий (≈5,9 с проти 2,85 с) – саме та картина, яку описує методичка.

4. Наскільки різняться три повтори одного виміру? Чи достатньо було б одного запуску?

Відповідь: TTFT локальної моделі стабільний (2,079–2,118 с), а загальний час на довгому промпті – 6,36–11,00 с, бо довжина відповіді щоразу різна (1469–2039 символів). TTFT хмари коливався в 2,4 раза (0,21–0,50 с) через мережу й чергу. Одного запуску недостатньо – потрібні повтори й медіана.

5. Чому холодний старт повільніший за теплий?

Відповідь: Під час холодного старту Ollama має прочитати ваги моделі з диска, розмістити їх у відеопам'яті й ініціалізувати рантайм – у нас це додало ≈2 с (4,05 проти 2,07 с). При теплому старті модель уже в пам'яті, і TTFT складається лише з з'єднання та prefill.

Відповіді на питання (крок 7, чат-бот Claude)

1. Що фізично відбувається під час TTFT — часу до першого токена — і чому цей показник вимірюють окремо від загальної швидкості генерації?

Відповідь: За TTFT встигає відбутися: встановлення з'єднання й передача запиту, черга на сервері, токенізація, за потреби завантаження моделі в пам'ять і prefill – прохід усього промпта через модель із заповненням KV-кешу, після чого генерується перший токен. Це окрема фаза з іншими обмеженнями, ніж генерація, і саме вона визначає, скільки користувач дивиться на порожній екран.

2. Чому хмарна модель може мати гірший TTFT, але кращу швидкість генерації, ніж локальна?

Відповідь: TTFT хмарної моделі включає мережу (DNS, TLS, сотні кілометрів до дата-центру) і чергу на спільному сервері. Зате далі генерація йде на потужному прискорювачі з дуже швидкою пам'яттю, тож токенів за секунду в рази більше, ніж на ноутбучному GPU.

3. Чому перший запуск локальної моделі повільніший за наступні і що таке холодний старт?

Відповідь: Холодний старт – запит, коли моделі ще немає в пам'яті: Ollama має прочитати гігабайти ваг із диска, розмістити їх у відеопам'яті й ініціалізувати рантайм. Наступні запити (теплий старт) застають модель уже завантаженою, поки вона не вивантажиться після тайм-ауту бездіяльності.

4. Чому при вимірюванні продуктивності беруть медіану кількох запусків, а не одне значення?

Відповідь: Окремий вимір зашумлений: фонові процеси, кеші, мережа, різна довжина відповіді. Медіана стійка до викидів (одного аномально повільного запуску), на відміну від середнього, а кілька повторів показують, наскільки результат стабільний.

5. Чому обробка вхідного запиту йде швидше, ніж генерація відповіді такої ж довжини?

Відповідь: Під час prefill усі токени промпта обробляються паралельно однією великою матричною операцією, і GPU завантажений обчисленнями. Генерація ж послідовна: кожен новий токен залежить від попереднього, і на кожен

треба заново прочитати всі ваги з пам'яті, тож вона обмежена пропускною здатністю пам'яті.

Висновок за завданням 4

Для інтерактивного чат-бота важливіший TTFT: користувач оцінює швидкість за тим, як скоро з'явилося перше слово, а далі текст «друкується» стрімінгом. Для фонової обробки великих обсягів тексту важливіша швидкість генерації (і вартість токена): TTFT там губиться на тлі хвилин генерації. Головний урок цього завдання – одне число затримки бреше: без розкладу на з'єднання, prefill і генерацію ми б вирішили, що локальна модель повільно «думає», хоча 2 с з'їдав мережевий стек Windows.

Завдання 5. Оцінювання якості моделей за рубрикою

Набір завдань складено до запуску моделей (quality.py), варіант 7 («Кіно»): 1 – рік першого публічного кінопоказу братів Люм'єр (правильно: 1895, 28 грудня, Гран-кафе, Париж; джерело – uk.wikipedia.org/wiki/Брати_Люм'єр); 2 – JSON event, year, city; 3 – стиснення абзацу про Одеську кіностудію (6 речень, числа 1907, 1919, 350, 700, 2015, 50 %+1, 2019, 100; джерело – uk.wikipedia.org/wiki/Одеська_кіностудія); 4 – функція top_rated(films, n); 5 – зал 180 місць, сеанс 1 год 45 хв + 15 хв перерви, 10:00–23:00 (правильно: 6 сеансів, 1080 глядачів); 6 – чим документальне кіно відрізняється від художнього. Об'єктивні перевірки (json.loads і запуск коду на тестових прикладах) виконує check_quality.py.

Кожне завдання прогнано двічі (temperature = 0,7) для трьох моделей; відповіді – у quality.json. Бал у табл. 13 – менший із двох прогонів (у дужках – бали кожного прогону): якщо модель хоча б раз провалила завдання, покладатися на неї в ньому не можна.

Таблиця 13 – Оцінка якості за рубрикою

Завдання	**llama3.2:3b**	**qwen3:4b**	**gpt-oss-120b (хмарна)**
1. Факт	0 (0/0): 1903, 1915	2 (2/2): 1895, 1895	2 (2/2): 1895, 1895
2. Формат	2 (2/2)	2 (2/2)	2 (2/2)
3. Стиснення	0 (0/0): загублено 6 із 8 чисел	1 (1/1): числа є, але «заснована в 1907»	2 (2/2)
4. Код	2 (2/2)	2 (2/2)	2 (2/2): код у ```python
5. Логіка	0 (0/0)	0 (2/0)	2 (2/2)
6. Українська	0 (1/0)	1 (1/1)	1 (1/2)
Разом /12	4	8	11

Таблиця 14 – Розбіжність оцінок між двома прогонами (так / ні)

Завдання	**llama3.2:3b**	**qwen3:4b**	**gpt-oss-120b**
1. Факт	ні (але роки різні: 1903 / 1915)	ні	ні
2. Формат	ні (але в прогоні 2 вигаданий рік 1936)	ні	ні
3. Стиснення	ні	ні	ні
4. Код	ні	ні	ні
5. Логіка	ні	так (6 сеансів → 4)	ні
6. Українська	так	ні	так

Пояснення оцінок. Завдання 2: усі шість відповідей json.loads() розібрав без правок; у llama (прогін 2) – валідний JSON із вигаданою подією «Фестиваль кіно Люм'єр» 1936 року, але рубрика завдання 2 оцінює лише формат. Завдання 3: llama залишила лише 1907 і 1919 (прогін 1) або 1907 і 350 з російським вступом «Читаем текст:» (прогін 2); qwen3 зберегла всі вісім чисел, але переписала «веде свою історію з 1907 року» як «заснувана в 1907 році», що суперечить ювілею 2019 року (додано інформацію, якої в тексті не було). Завдання 4: усі функції пройшли тест check_quality.py; gpt-oss-120b обидва рази обгорнула код у ```python і в прогоні 1 написала docstring російською, тоді як малі моделі дали «голий» код. Завдання 5: llama – безглузді обчислення (13 сеансів) і зациклений текст; gpt-oss-120b обидва рази – 6 сеансів і 1080 глядачів із перевіркою розкладом. Завдання 6: llama – русизми («беспристрастним»), латиниця посеред слова («epotelізацій»), один абзац; qwen3 – кальки («Керівник фільму», «фіктивних», «споживачів») і зіпсований символ «�»; gpt-oss-120b – природна мова, але в прогоні 1 некоректний термін «художнє (фантастичне) кіно».

Відповіді на питання (крок 6)

1. Яка модель набрала найбільше балів і на яких саме завданнях виникла основна різниця?

Відповідь: Хмарна gpt-oss-120b – 11 із 12, далі qwen3:4b – 8 і llama3.2:3b – 4. Основна різниця – на фактах (1), стисненні з числами (3) і багатокроковій логіці (5); формат JSON (2) і простий код (4) усі три моделі виконали однаково добре.

2. На яких завданнях оцінки двох прогонів розійшлися? Що це говорить про надійність моделі?

Відповідь: У qwen3:4b на логіці (2 → 0), у llama і хмарної моделі – на українській мові; ще в llama відповіді розходилися за змістом при однаковому балі (роки 1903/1915, у JSON – 1895/1936). Отже, одного вдалого прогону недостатньо: та сама модель може розв'язати задачу й провалити її при повторі.

3. Чи виконала хоч одна модель вимогу «без markdown» у завданні 2?

Відповідь: Так, усі три й в обох прогонах: шість із шести відповідей розібрано json.loads() без правок. Але дисципліна формату не стала загальною: у завданні 4 («Лише код») хмарна модель двічі обгорнула код у ```python, а валідний JSON llama містив вигаданий рік.

4. Чи були проблеми з українською мовою — русизми, кальки, перехід на іншу мову?

Відповідь: Так: у llama – русизми, російські вставки («Читаем текст:», розв'язки російською) і латиниця посеред слів; у qwen3 – кальки («заснувана», «фіктивних», «Керівник фільму») і зіпсований символ; хмарна модель писала природною українською, з однією термінологічною неточністю й російським docstring у коді.

Приклад провалу (крок 7): qwen3:4b, завдання 5, прогін 2

Запит: «Зал 180 місць, сеанс 1 год 45 хв + 15 хв перерва, робота з 10:00 до 23:00. Скільки сеансів і глядачів? Покажи хід розв'язання.»

Відповідь моделі повністю, як у quality.json (довгі рядки перенесено лише для друку):

```
Для визначення кількості сеансів, які можна провести від 10:00 до 23:00 з урахуванням того, що кожен
сеанс триває 1 година 45 хвилин, а після кожного (крім останнього) відбувається перерва 15 хвилин,
розрахуємо так:

1. **Перший сеанс**: 10:00 – 12:45
2. **Перерва**: 12:45 – 13:00
3. **Другий сеанс**: 13:00 – 15:45
4. **Перерва**: 15:45 – 16:00
5. **Третій сеанс**: 16:00 – 18:45
6. **Перерва**: 18:45 – 19:00
7. **Четвертий сеанс**: 19:00 – 21:45
8. **Перерва**: 21:45 – 22:00
9. **П'ятий сеанс**: 22:00 – 23:45 (завершується після 23:00, тому недопустимо)

Робота закінчується на 23:00, а п'ятий сеанс завершується в 23:45, що виходить за межі. Значить,
максимальна кількість сеансів, які можна провести без порушення умов, – **4**.

**Відповідь**: 4 сеанса.
```

Правильна відповідь: 13 год = 780 хв; n сеансів потребують 105n + 15(n − 1) хв ≤ 780 → n ≤ 6,6, тобто 6 сеансів (10:00–11:45, 12:00–13:45, …, 20:00–21:45) і 6 × 180 = 1080 глядачів.

Чому так сталося: модель помилилася в першому ж арифметичному кроці – додала до 10:00 не 1 год 45 хв, а 2 год 45 хв («10:00 — 12:45»). Далі кожен рядок розкладу послідовно спирався на попередній, тож помилка розмножилася й дала 4 сеанси замість 6, а друге питання («скільки глядачів») модель узагалі загубила. Відповідь виглядає переконливо – акуратний розклад, перевірка межі 23:00 – і саме це небезпечно: правдоподібність міркування не означає його правильності. У прогоні 1 та сама модель склала нерівність $105n + 15(n - 1) \leq 780$ і отримала 6 сеансів і 1080 глядачів: результат залежить від того, яку стратегію (формула чи «покроковий розклад») модель випадково обрала на початку.

Відповіді на питання (крок 8, чат-бот Claude)

1. Я оцінював мовні моделі за рубрикою з шести завдань і трибальною шкалою. Які ще підходи до оцінювання якості мовних моделей використовують на практиці?

Відповідь: Стандартизовані бенчмарки (MMLU, GSM8K, HumanEval тощо), автоматичні метрики для перевірюваних задач (точний збіг, проходження тестів, json-валідація), LLM-as-a-judge – оцінювання іншою моделлю за рубрикою, попарні порівняння людьми (Chatbot Arena, Elo), а також A/B-тести та моніторинг на реальних користувачах.

2. Чому мовні моделі часто ігнорують пряму інструкцію не використовувати markdown при генерації JSON?

Відповідь: У навчальних даних і при донавчанні чат-моделей JSON і код майже завжди обгорнуті в ```json – ця звичка закріплена дуже сильно й «перемагає» одне речення інструкції. Надійно формат забезпечують structured output (обмеження генерації схемою) і валідація з повторним запитом.

3. Чому малі мовні моделі гірше пишуть українською, ніж англійською?

Відповідь: Українських текстів у навчальних корпусах на порядки менше, ніж англійських, і помітно менше, ніж російських; токенізатор ріже українські слова на дрібніші шматки. Мала модель не має достатньо параметрів, щоб якісно вивчити рідкісну мову, тому з'являються русизми, кальки й перехід на іншу мову.

4. Чому та сама модель на однаковий запит може дати відповіді різної якості при повторному запуску?

Відповідь: Бо при temperature > 0 кожен наступний токен вибирається випадково з розподілу; ранній «невдалий» вибір змінює весь подальший текст, і помилка накопичується. Тому повторні прогони показують не збій, а розкид якості, який сам є характеристикою моделі.

5. Що таке галюцинація мовної моделі і чому модель радше вигадає факт, ніж визнає незнання?

Відповідь: Галюцинація – впевнено сформульована, але неправдива відповідь. Модель навчена продовжувати текст найправдоподібніше, а не перевіряти факти; у навчальних даних питання майже завжди супроводжуються відповіддю, а не «не знаю», і винагорода при донавчанні часто заохочує «корисну» відповідь. Тому правдоподібний рік для моделі ймовірніший за визнання незнання.

Висновок за завданням 5

Оцінка за рубрикою виявилася надійною там, де перевірка об'єктивна (json.loads, запуск коду, звірка чисел і року з джерелом), і менш надійною для мови (завдання 6), де межа між 1 і 2 балами залежить від оцінювача. Найпоказовішими для порівняння моделей стали завдання 1 (факт), 3 (числа в переказі) і 5 (логіка) – саме на них розійшлися моделі й прогони; завдання 2 і 4 в нашому наборі всі моделі пройшли. Для роботи з українськомовним текстом обрав би хмарну gpt-oss-120b; якщо дані не можна передавати назовні – qwen3:4b із обов'язковою перевіркою результату, але не llama3.2:3b.

Завдання 6. Керування випадковістю: temperature і top_p

Модель – llama3.2:3b (провайдер local). Варіант 7: творче завдання – «Назва застосунку обліку особистих витрат», фактичне питання – «Рік першого кінопоказу братів Люм'єр». Правильна відповідь – 1895 рік (28 грудня 1895 р., Гран-кафе на бульварі Капуцинок, Париж; джерело: uk.wikipedia.org/wiki/Брати_Люм'єр).

Таблиця 16 – Вплив параметрів на різноманітність відповідей

Параметр	**Значення**	**Унікальних із 5 (творче)**	**Правильних із 5 (фактичне)**
temperature	0.0	2	0 (усі 5 – «1998»)
temperature	0.7	5	—
temperature	1.5	5	0 (9 років, 1845, 1943, 1978, відмова)
top_p (t = 0.7)	0.1	1	—
top_p (t = 0.7)	0.5	5	—
top_p (t = 0.7)	1.0	5	—

Таблиця 17 – Перевірка відтворюваності

Показник	**Значення**
Кількість запусків	10
temperature	0.0
Унікальних відповідей	1 («Рік першого кінопоказу братів Люм'єр – 1998 рік.»)
Усі відповіді однакові (так / ні)	так (але всі хибні)

Творче завдання модель зрозуміла як «назви список назв», тому кожна відповідь – перелік із 10 англомовних назв. При temperature = 0 перша відповідь відрізнялася від решти чотирьох, ідентичних між собою (див. питання 4). При temperature = 1.5 відповіді змішують українську, англійську, німецьку й російську і подекуди перетворюються на набір псевдослів.

Відповіді на питання (крок 7)

1. Яке значення температури ви поставите агенту, який має повертати JSON, і чому?

Відповідь: 0 (у крайньому разі 0,1–0,2): агентові потрібна передбачуваність і стабільний формат, а не різноманітність. Наш експеримент показав, що вже при 1,5 модель ламає навіть мову, не кажучи про синтаксис.

2. Яке — генератору варіантів заголовків?

Відповідь: Близько 0,7–0,9: при 0,7 ми отримали 5 різних і ще читабельних варіантів із 5, а при 1,5 – різноманітні, але значною мірою безглузді.

3. Чим вплив top_p відрізняється від впливу temperature за характером, а не за силою?

Відповідь: Temperature підсилює ймовірність маловірогідних токенів, тому на високих значеннях з'являється сміття. top_p лише відрізає «хвіст»: при 0,1 залишилися тільки найімовірніші токени, і всі п'ять відповідей збіглися (як жадібний вибір), а при 0,5–1,0 відповіді різні, але граматичні – нових маловірогідних токенів top_p не додає.

4. Чи означає temperature=0 повну відтворюваність результату? Що показав ваш експеримент із 10 запусків?

Відповідь: Ні. На фактичному питанні всі 10 відповідей збіглися, але на творчому при temperature = 0 вийшло 2 унікальні з 5: перший запуск обчислював промпт повністю, а наступні брали його з кешу (Ollama кешує префікс промпта), шлях обчислень з плаваючою комою відрізнявся, і тексти розійшлися вже в першому реченні. До того ж відтворюваність ≠ правильність: 10 однакових відповідей «1998» – стабільна галюцинація.

5. Наведіть найбезглуздішу відповідь, яку модель видала при temperature=1.5, і поясніть, чому вона взагалі могла з'явитися.

Відповідь: «Перший рік кінопоказу братлів Люм’єр почнеш у 9 років.» (а також «Включені найми possible Namen eines budgets-ähnlichen Aplications…»). При T = 1,5 розподіл сплющується, і токени з інших мов чи несумісні за змістом отримують помітну ймовірність; щойно один такий токен обрано, наступні генеруються вже з урахуванням зіпсованого контексту, і помилка лавиноподібно наростає.

Відповіді на питання (крок 8, чат-бот Claude)

1. Що саме математично робить параметр temperature із розподілом ймовірностей наступного токена?

Відповідь: Логіти діляться на T перед softmax: $p_i = \exp(z_i / T) / \Sigma \exp(z_j / T)$. При $T < 1$ розподіл загострюється (найімовірніший токен домінує), при $T > 1$ – сплющується, а $T \to 0$ вироджується в жадібний вибір argmax.

2. Чим top_p відрізняється від temperature — обидва впливають на різноманітність, але за яким механізмом?

Відповідь: Temperature змінює форму всього розподілу, не відкидаючи жодного токена; top_p (nucleus sampling) не змінює співвідношень, а відрізає «хвіст»: лишає найменший набір токенів із сумарною ймовірністю $\geq p$ і вибирає лише серед них.

3. Чи є сенс змінювати обидва параметри одночасно, чи краще налаштовувати щось одне?

Відповідь: Зазвичай радять змінювати один параметр, а другий лишати за замовчуванням: їхні ефекти накладаються і важко зрозуміти, що дало результат. Комбінація має сенс, коли потрібна висока температура для різноманітності, а $top_p < 1$ – як запобіжник від зовсім абсурдних токенів.

4. Чому навіть при temperature=0 мовна модель не гарантує абсолютно однакову відповідь при повторних запусках?

Відповідь: Обчислення з плаваючою комою на GPU неасоціативні: порядок підсумовування залежить від паралелізації, батчів і кешу, тож логіти двох майже рівних токенів можуть «помінятися місцями». Крім того, сервер може по-різному обробляти контекст (кеш промпта, різні ядра), і першого ж різного токена досить, щоб далі текст розійшовся.

5. Які значення temperature використовують на практиці для витягу структурованих даних, а які для генерації творчого тексту?

Відповідь: Для витягу даних, JSON, класифікації й виклику інструментів – 0–0,2 (передбачуваність важливіша за різноманітність). Для творчого тексту, назв і брейнштормінгу – 0,7–1,0, іноді до 1,2; вище 1,3 текст швидко стає безладним.

Висновок за завданням 6

Для наступних лабораторних, де модель повертає структуровані дані й викликає інструменти, обираю temperature = 0 (або 0,1–0,2) і top_p = 1, змінюючи лише один параметр. Налаштування «за замовчуванням» (у Ollama temperature = 0,8) непридатне: воно навмисно додає випадковість, а отже різний формат і різні факти від запуску до запуску. Але й temperature = 0 не робить модель правою – вона лише стабільно повторює ту саму відповідь, тому факти треба брати з інструментів або документів, а формат – перевіряти в коді.

Підсумкове порівняння моделей (крок 8)

Таблиця 18 – Підсумкове порівняння моделей

Критерій	**Локальна мала (3B): llama3.2:3b**	**Локальна більша (4B): qwen3:4b**	**Хмарна: gpt-oss-120b (Groq)**
Швидкість генерації	≈110 токенів/с (табл. 2); 230 симв./с на довгому промпті (табл. 7)	≈89 токенів/с (табл. 2), але з роздумами відповідь у рази довша	≈480 токенів/с; 1173 симв./с на довгому промпті
TTFT	2,1 с через localhost, 0,05 с через 127.0.0.1 (табл. Д3)	перший видимий токен – лише після роздумів: 13–55 с на фактичному питанні	0,26–0,50 с (мережа + черга + короткий reasoning)
Якість, балів /12	4	8	11
Стабільність формату (завдання 2)	2/2 прогони – валідний JSON (але 1 раз вигаданий рік)	2/2 – валідний JSON	2/2 – валідний JSON (але код обгортає в markdown)
Потрібні ресурси	2,0 ГБ на диску, ≈2,6 ГБ VRAM; працює й на CPU	2,5 ГБ на диску, ≈3,2 ГБ VRAM; на CPU дуже повільно через роздуми	лише інтернет і API-ключ; обчислення на залізі провайдера
Вартість	безкоштовно (електроенергія)	безкоштовно (електроенергія)	безкоштовний тариф із лімітами (8000 токенів/хв, 1000 запитів/добу), далі – оплата за токени
Куди йдуть ваші дані	залишаються на комп'ютері	залишаються на комп'ютері	на сервери провайдера (Groq) за кордоном
Працює без інтернету	так	так	ні
Коли обирати саме її	прості масові задачі без вимог до фактів і мови (класифікація, витяг із наданого тексту, шаблонний код), коли потрібна швидка відповідь і дані не можна передавати	локальні задачі з міркуванням (арифметика, перевірка умов), коли дані конфіденційні, а час очікування не критичний; з перевіркою результату	коли важливі факти, логіка, українська мова й швидкість довгих відповідей, а дані можна передавати назовні; з лімітами й резервним провайдером

Правило вибору: локальну модель беру, якщо дані не можна передавати третій стороні, потрібна робота без інтернету або нульова гранична вартість великих обсягів – і задачу можна обмежити до того, з чим мала модель справляється (робота з наданим текстом, формат під програмною перевіркою). Хмарну беру, коли важливі якість, знання й мова, а дані не конфіденційні; при цьому закладаю ліміти запитів, резервного провайдера (той самий OpenAI-сумісний код) і не надсилаю персональних даних.

Загальні висновки

1) Локальна модель на GPU ноутбука працює як звичайний веб-сервер з OpenAI-сумісним API: керування з коду, журналювання й вимірювання не відрізняються від хмарного провайдера – перехід на Groq зводиться до зміни base_url, ключа й назви моделі;

2) Хмарна gpt-oss-120b дала 11 балів із 12 проти 8 (qwen3:4b) і 4 (llama3.2:3b) та в 5 разів вищу швидкість генерації на довгих відповідях, але ціною

передачі даних назовні, залежності від мережі й лімітів безкоштовного тарифу (8000 токенів/хв);

3) Затримка складається з двох незалежних показників – TTFT і швидкості генерації; навіть TTFT локальної моделі на 97 % може складатися з мережевої дрібниці (відкат localhost з IPv6 на IPv4), тому висновок «яка модель швидша» без розкладу затримки некоректний;

4) Режим міркування окупається на багатокроковому виведенні (5/5 проти 2/5) і є перевитратою на фактичних питаннях (у 612 разів довше за той самий зміст);

5) Промпт і system-повідомлення – ненадійний засіб керування форматом і мовою: мала модель порушила мовну інструкцію, міркувальна – вимогу «одним словом»; надійність досягається програмною валідацією;

6) Мовні моделі недетерміновані навіть при temperature = 0, а стабільна відповідь може бути стабільною галюцинацією, тому вимірювання потребують повторів, а факти – зовнішнього джерела.